

Exp 9: CAN Communication

Name: Gobinda Dey
CDSID: GDEY



Table of Contents

S.No.		Page No
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	3
5	Working Principle	4
6	Application Code	4
7	Output	5
8	Hardware Image	5
9	Appendix	6

1. Introduction:

- Controller Area Network (CAN) is a robust serial communication protocol widely used in automotive and industrial applications. It enables multiple microcontrollers to communicate with each other without a host computer.
- In this experiment, CAN communication is implemented using the Raspberry Pi Pico and MCP2515 CAN controller module. Two nodes are created: one acts as a transmitter and the other as a receiver. Data is transmitted from one Pico board to another via CAN bus and displayed on a serial console.

2. Scope:

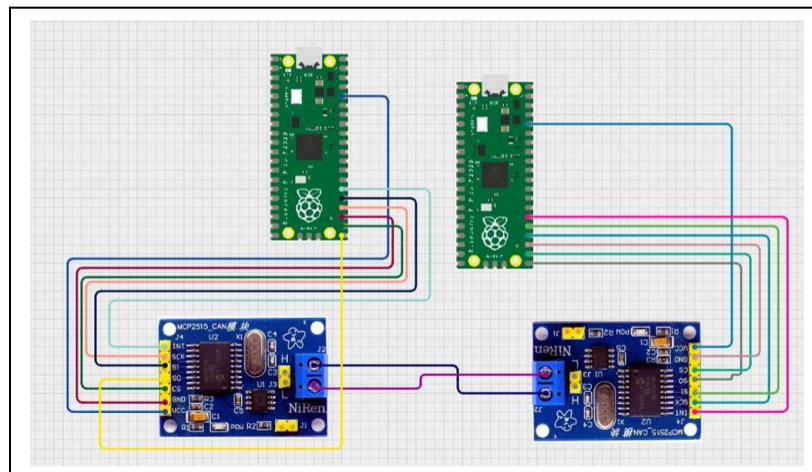
- To understand CAN protocol fundamentals
- To interface MCP2515 CAN module with Raspberry Pi Pico using SPI
- To implement transmitter and receiver communication
- To monitor real-time CAN data using serial terminal

3. Prerequisite:

Hardware: Raspberry Pi Pico Microcontroller (2 units), USB Connector (2 units), Breadboard (2 units), MCP2515 CAN Module (2 units), 120Ω resistor

Software: Visual studio code, PuTTY Serial Console, Pico SDK, CMake & GCC Toolchain

4. Connection Diagram:



5. Working Principle:

- The MCP2515 module acts as a CAN controller and communicates with the Raspberry Pi Pico using SPI protocol.
- The transmitter Pico reads input from the serial terminal (PuTTY).
- The input character is sent to MCP2515 via SPI.
- MCP2515 converts data into CAN frame and transmits it on CAN bus.
- The receiver MCP2515 detects the frame and stores it in RX buffer.
- Receiver Pico reads the data via SPI.
- The received data is displayed on serial terminal.

6. Application code:

Transmitter Code:

```
Assignment_9.c > main()
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "hardware/spi.h"
4
5  #define SPI_PORT spi0
6  #define CS_PIN 17
7
8  void mcp2515_reset() {
9      gpio_put(CS_PIN, 0);
10     spi_write_blocking(SPI_PORT, (uint8_t[]){0xC0}, 1);
11     gpio_put(CS_PIN, 1);
12 }
13
14 void mcp2515_write_register(uint8_t reg, uint8_t value) {
15     uint8_t buf[3] = {0xC2, reg, value};
16     gpio_put(CS_PIN, 0);
17     spi_write_blocking(SPI_PORT, buf, 3);
18     gpio_put(CS_PIN, 1);
19 }
20
21 void can_send() {
22     uint8_t data[] = {0x55};
23
24     gpio_put(CS_PIN, 0);
25     uint8_t buf[] = {
26         0x40, // Load TX buffer
27         0x00, 0x00, // ID
28         0x01, // DLC
29         data[0]
30     };
31     spi_write_blocking(SPI_PORT, buf, sizeof(buf));
32     gpio_put(CS_PIN, 1);
33
34     gpio_put(CS_PIN, 0);
35     uint8_t rts = 0xB1; // Request to send
36     spi_write_blocking(SPI_PORT, &rts, 1);
37     gpio_put(CS_PIN, 1);
38 }
39
```

```
int main() {
    stdio_init_all();

    spi_init(SPI_PORT, 1 * 1000 * 1000);
    gpio_set_function(16, GPIO_FUNC_SPI);
    gpio_set_function(19, GPIO_FUNC_SPI);
    gpio_set_function(18, GPIO_FUNC_SPI);

    gpio_init(CS_PIN);
    gpio_set_dir(CS_PIN, GPIO_OUT);
    gpio_put(CS_PIN, 1);

    sleep_ms(1000);
    mcp2515_reset();

    while (1) {
        can_send();
        printf("Message Sent\n");
        sleep_ms(1000);
    }
}
```

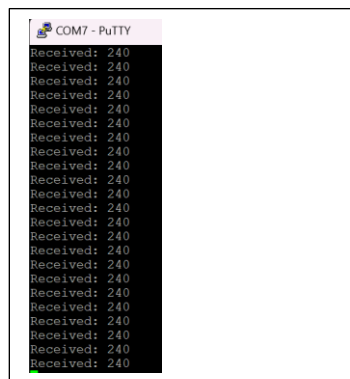
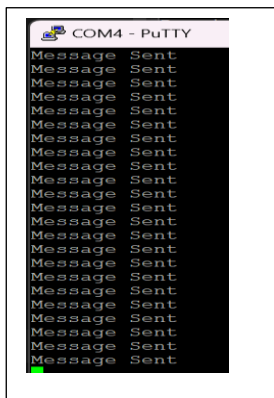
Receiver Code:

```

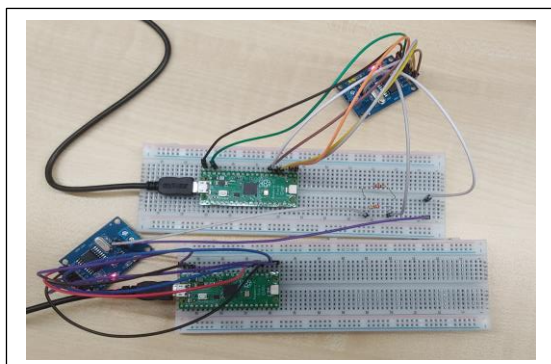
1  # Assignment 9.c 2.7 main()
2  #include <stdio.h>
3  #include "pico/stdlib.h"
4  #include "hardware/spi.h"
5
6  #define SPI_PORT spi0
7  #define CS_PIN 17
8
9  void mcp2515_reset() {
10     gpio_put(CS_PIN, 0);
11     spi_write_blocking(SPI_PORT, (uint8_t[]){0xc0}, 1);
12     gpio_put(CS_PIN, 1);
13 }
14
15 void mcp2515_write_register(uint8_t reg, uint8_t value) {
16     uint8_t buf[3] = {0x02, reg, value};
17     gpio_put(CS_PIN, 0);
18     spi_write_blocking(SPI_PORT, buf, 3);
19     gpio_put(CS_PIN, 1);
20 }
21
22 void can_send() {
23     uint8_t data[] = {0x55};
24
25     gpio_put(CS_PIN, 0);
26     uint8_t buf[] = {
27         0x40, // Load TX buffer
28         0x00, 0x00, // ID
29         0x01, // DLC
30         data[0]
31     };
32     spi_write_blocking(SPI_PORT, buf, sizeof(buf));
33     gpio_put(CS_PIN, 1);
34
35     gpio_put(CS_PIN, 0);
36     uint8_t rts = 0x81; // Request to send
37     spi_write_blocking(SPI_PORT, &rts, 1);
38     gpio_put(CS_PIN, 1);
39 }

```

7. Output:



8. Hardware Image:



9. Appendix:

References

YouTube and Other Online Sources

[Official Documentation for Raspberry Pi Pico and Pico W](#)

[GitHub Repository for Raspberry Pi Pico SDK](#)